



Machine Learning: From Theory to Practice

A hands-on guide for professionals and developers — from core concepts to production-ready code.



What is Machine Learning?

Algorithms that **learn patterns from data** without explicit programming — no hand-crafted rules, just learned intelligence.

Fraud Detection

Real-time anomaly detection across millions of transactions per second

Medical Diagnosis

Detecting cancer, reading scans, predicting patient outcomes

Autonomous Vehicles

Perception, path planning, and real-time decision-making at scale

Recommendation Systems

Netflix, Spotify, Amazon — surfacing what you want before you ask

ML

VS

**Expert
system**



AI vs Machine Learning vs Deep Learning



Artificial Intelligence (AI)

The broad concept of machines perform tasks intelligently and including.

Example: Chatbots, recommendation systems



Machine Learning (ML)

A subset of AI that learns from data.

Example: Spam detection, predictions



Deep Learning (DL)

A subset of ML, that uses early neural networks.

Exc: Image recognition, voice assistants



Machine Learning v/s Deep Learning

Machine Learning



Data



Feature extraction



ML model



Decision

Deep Learning



Data



DL model



Decision

ML only models

- Linear Regression
- Logistic Regression
- Decision Trees
- Random Forest
- Support Vector Machines (SVM)
- K-Nearest Neighbors (KNN)
- Naive Bayes
- XGBoost, LightGBM
- K-Means Clustering
- DBSCAN
- Principal Component Analysis
- Hidden Markov Models (HMM)

DL only models

- Artificial Neural Networks (ANN)
- Convolutional Neural Networks (CNN)
- Recurrent Neural Networks (RNN)
- Long Short-Term Memory (LSTM)
- Gated Recurrent Unit (GRU)
- Transformer Models (e.g., BERT, GPT)
- Autoencoders
- Generative Adversarial Networks (GANs)
- Deep Belief Networks (DBN)
- Graph Neural Networks (GNNs)
- Neural ODEs
- Physics-Informed Neural Networks (PINNs)

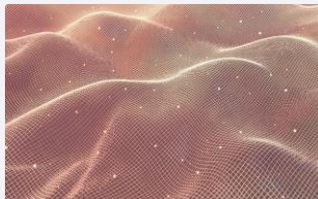
Three Types of Machine Learning



Supervised Learning

Learn from **labeled data** — input/output pairs teach the model a mapping function.

- Predicting house prices from features
- Email spam classification
- Credit risk scoring



Unsupervised Learning

Discover **hidden patterns** in unlabeled data — the model finds structure on its own.

- Customer segmentation
- Anomaly detection
- Dimensionality reduction



Reinforcement Learning

Learn through **trial and error** — agents maximize cumulative reward signals.

- Game AI (AlphaGo, OpenAI Five)
- Robot control systems
- Chatbot response tuning

Activity 1: Identify the ML Type

INTERACTIVE QUIZ

Classify each scenario before revealing the answer. Think carefully — the framing matters.

Scenario A

Detecting fraudulent credit card transactions — historical fraud labels are provided in the training dataset.

Scenario B

Grouping Netflix users by viewing habits — no predefined categories or labels exist.

Scenario C

Training a chatbot to improve its responses based on positive and negative user feedback signals.

Answers

A → **Supervised**

Labeled fraud history drives classification.

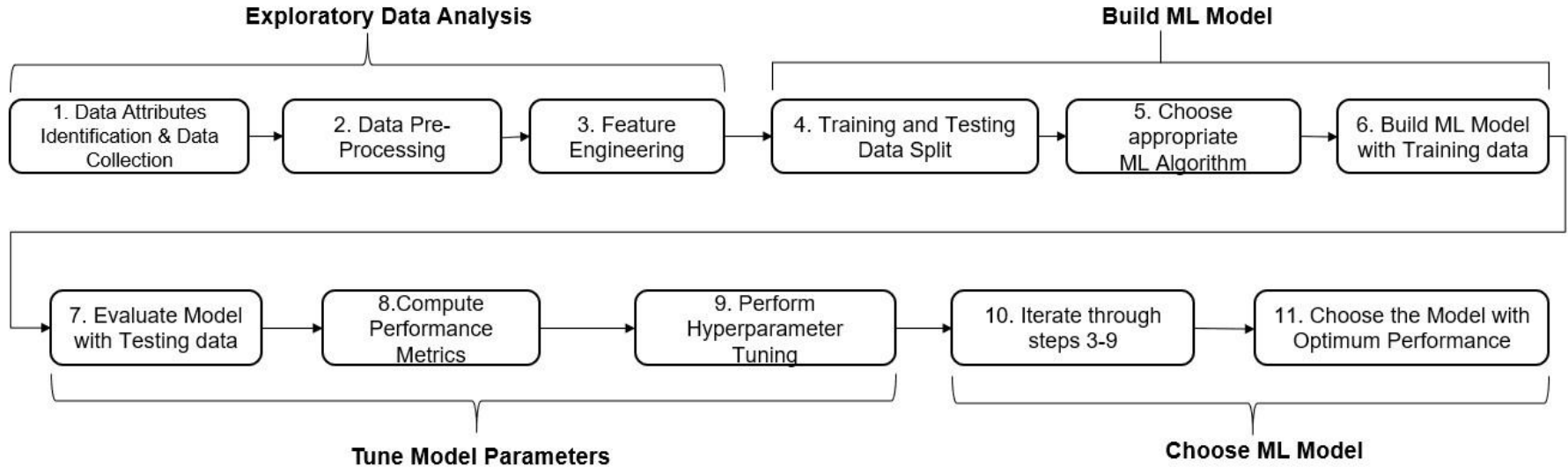
B → **Unsupervised**

No labels — clustering finds natural user groups.

C → **Reinforcement**

Reward signals (user feedback) guide policy improvement.

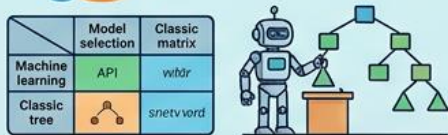
Machine Learning Pipeline



ESSENTIAL LIBRARIES FOR APPLIED MACHINE LEARNING

MODELING & CLASSIC ML

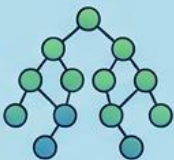
 **Scikit-Learn**



Comprehensive Traditional Classifier ML Algorithms

 **XGBoost**

High-Performance Gradient Boosting



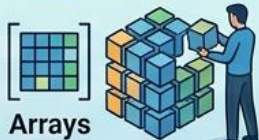
Matplotlib & Seaborn



Data Visualization

FOUNDATIONS: DATA MANIPULATION & ANALYSIS

 **NumPy**



Arrays

Efficient Array & Numerical Ops

 **pandas**



Data Analysis & Tables (DataFrames)

SUPPORTING TOOLS & DEPLOYMENT

 **OpenCV**



OpenCV
Computer Vision

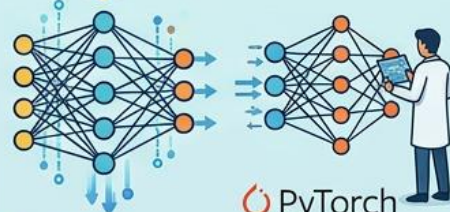
NLTK & spaCy



Natural Language Processing

DEEP LEARNING & NEURAL NETS

 **TensorFlow**



Scalable Learning Platform

 **PyTorch**
Dynamic Computation & Research

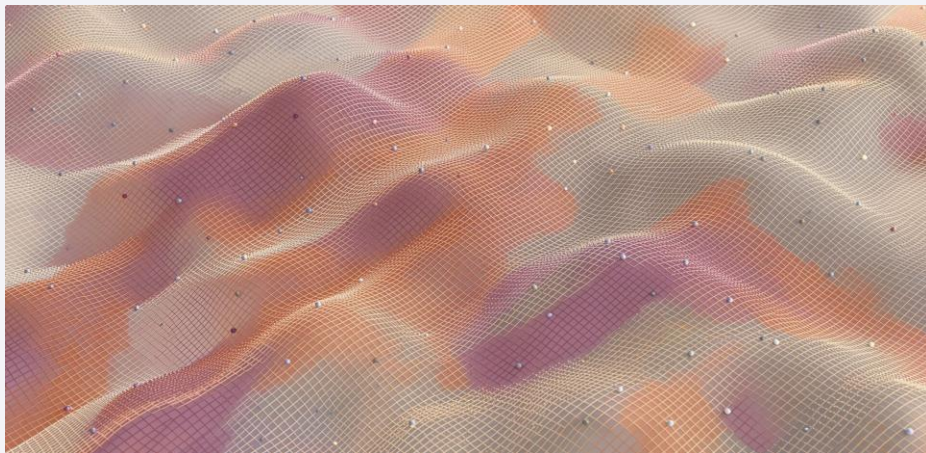
mlflow



Experiment Tracking & ML Ops

BUILDING PRACTICAL AI APPLICATIONS

NumPy: Fast Numerical Computing



Why NumPy?

Python lists are flexible — but for ML workloads, they're far too slow. NumPy's **N-dimensional arrays** are the backbone of every major ML framework.

50–100× Faster Than Loops

Vectorized C-level operations on 1M elements instead of Python iteration.

Vectorized Arithmetic

```
revenue = prices * quantities
```

— one line, no loop required.

ML Foundation

PyTorch, TensorFlow, and scikit-learn all build on NumPy arrays internally.

Activity 2: NumPy Hands-On Exercise

CODE CHALLENGE — 10 MIN

</>

Create arrays

1. Create arrays: `prices=[29.99, 49.99, 19.99]`,
`quantities=[1,2,3]`

Vectorize multiply

2. Calculate revenue with vectorized multiplication — no loops

Compute stats

3. Compute mean, std deviation, and filter items priced over
\$40

Reshape to 4×3

4. Reshape 12 sales values into a (4, 3) matrix (4 products × 3
months)

Scalar
(0D tensor)

- a single value

6

Vector
(1D tensor)

- row or column
of values

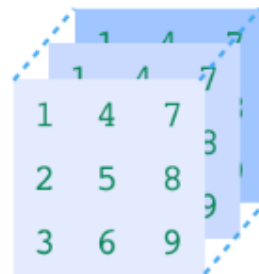
$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$ or $[1 \ 2 \ 3]$

Matrix
(2D tensor)

- array (rows
and columns)
of values

$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$

3D tensor, etc.



To write vectorized code in Python using NumPy, you must replace explicit Python loops (for and while) with direct operations on entire arrays

Element-Wise Math (Arithmetic & Functions)

```
# Loop Approach (Slow)
data = [1, 2, 3, 4]
squared_loop = [x ** 2 for x in data]
print("squared_loop",squared_loop)

# Vectorized Approach (Fast)
arr = np.array([1, 2, 3, 4])
squared_vec = arr ** 2           # Broad operators
log_vec = np.log(arr)           # Native ufuncs
print("squared_vec",squared_vec)
print("log_vec",log_vec)
```

Aggregations with the axis Parameter

```
matrix = np.array([[1, 2], [3, 4]])
# Sum everything into a single scalar
total = np.sum(matrix)
# Column-wise sum (collapses rows)
col_sum = np.sum(matrix, axis=0) # Output: [4, 6]
# Row-wise sum (collapses columns)
row_sum = np.sum(matrix, axis=1) # Output: [3, 7]
print(f"{total=}")
print(f"{col_sum=}")
print(f"{row_sum=}")
```

Indexing

- We can extract specific values from our tensors using indexing.
- Keep in mind that when indexing the row and column, indices start at 0.
- And like indexing with lists, we can use negative indices as well (where -1 is the last item).

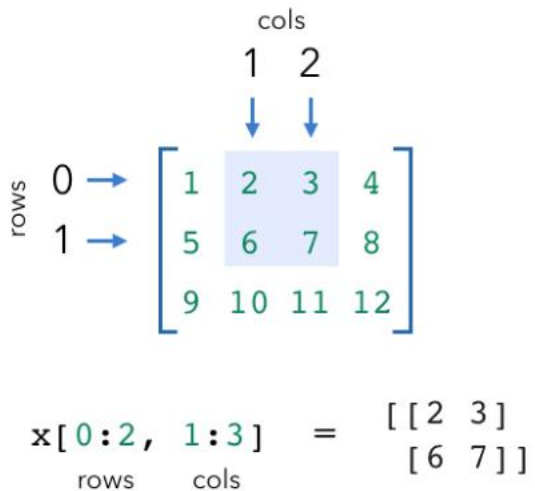


Diagram illustrating 2D array indexing. A 3x4 grid of numbers 1-12 is shown. Rows are indexed 0 to 2, and columns are indexed 1 to 2. A 2x2 subgrid (rows 0:2, cols 1:3) is highlighted in blue, containing values 2, 3, 6, and 7.

$$\begin{array}{c} \text{rows} \\ 0 \rightarrow \\ 1 \rightarrow \end{array} \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

cols
1 2
↓ ↓

$$\begin{array}{c} \text{rows} \\ \text{cols} \end{array} x[0:2, 1:3] = \begin{bmatrix} 2 & 3 \\ 6 & 7 \end{bmatrix}$$

3. Conditional Logic (Replacing if-else Loops)

Avoid looping through an array to flag or filter data based on conditions.

1. Use `np.where()` or boolean masking instead.
2. Boolean Filtering

```
grades = np.array([55, 82, 67, 90])
# Flagging: np.where(condition, value_if_true, value_if_false)
status = np.where(grades >= 60, "Pass", "Fail")
print(f"{status}")
# Boolean Filtering (Masking)
passing_grades = grades[grades >= 60] # Output: [82, 67, 90]
print(f"{passing_grades}")
```

4. Leveraging Broadcasting

Broadcasting allows you to perform operations on arrays of different shapes without manually duplicating data to force a fit.

```
# Adding a scalar to a 1D matrix
arr = np.array([10, 20, 30])
result = arr + 5 # 5 is "broadcasted" to [5, 5, 5] -> [15, 25, 35]

# Adding a 1D row vector to a 2D matrix
matrix = np.array([[1, 2, 3], [4, 5, 6]])
row_vector = np.array([1, 0, -1])
combined = matrix + row_vector # Automatically aligns column dimensions
print(f"{combined}")
```

In [NumPy](#), broadcasting allows arithmetic operations between arrays of different shapes by virtually expanding the smaller array to match the larger one.

NumPy Deep Dive: Vectorization & Performance

50–100×

Speedup

Vectorized ops vs. Python loops on
1M elements

1

Line of Code

Total revenue across 1M products:
`prices * quantities`

3

Core Techniques

Vectorization, Boolean indexing,
Broadcasting

Boolean Indexing

Filter arrays without a single loop:

```
premium = prices[prices > 40]  
high_revenue = sales[sales[:, 1] > 10000]
```

Broadcasting

Operate on arrays of different shapes automatically:

```
monthly = daily_sales * np.array([31, 28, 31])  
normalized = (data - data.mean()) / data.std()
```

Common NumPy activities include array creation, mathematical operations, and data manipulation.

Core NumPy Practice Activities

- **Array Creation:** Build 1D, 2D, or 3D arrays, identity matrices, and random number sequences.
- **Indexing & Slicing:** Extract specific rows, columns, or sub-arrays using index positions and boolean masks.
- **Shape Manipulation:** Change array dimensions using `reshape()`, flatten arrays, or join and split data.
- **Mathematical Operations:** Perform element-wise addition, multiplication, matrix dot products, and statistical aggregation (mean, median, standard deviation).
- **Broadcasting:** Apply operations on arrays of different sizes without writing explicit loops.

Pandas: From Raw Data to Clean Features

1

Load & Explore

Read CSV/Excel, inspect `.shape`, missing values, and correlations

2

Clean Data

Drop nulls, cap outliers, remove duplicates, filter invalid rows

3

Feature Engineering

GroupBy aggregations, create ratio features, encode categoricals

Think of Pandas DataFrames as **Excel spreadsheets supercharged with Python** — every data transformation you need in 5 lines or fewer.



Activity 4: Pandas Data Cleaning Pipeline

REAL-WORLD SCENARIO

Mission: Clean 1,500 raw candidate records and produce a model-ready DataFrame.

01

Load & Inspect

Read CSV, check `.shape` and `.isnull().sum()`

02

Drop Critical Nulls

Remove rows missing salary or experience — these can't be imputed

03

Cap Outliers

Clip salary to the \$20K–\$500K valid range using `.clip()`

04

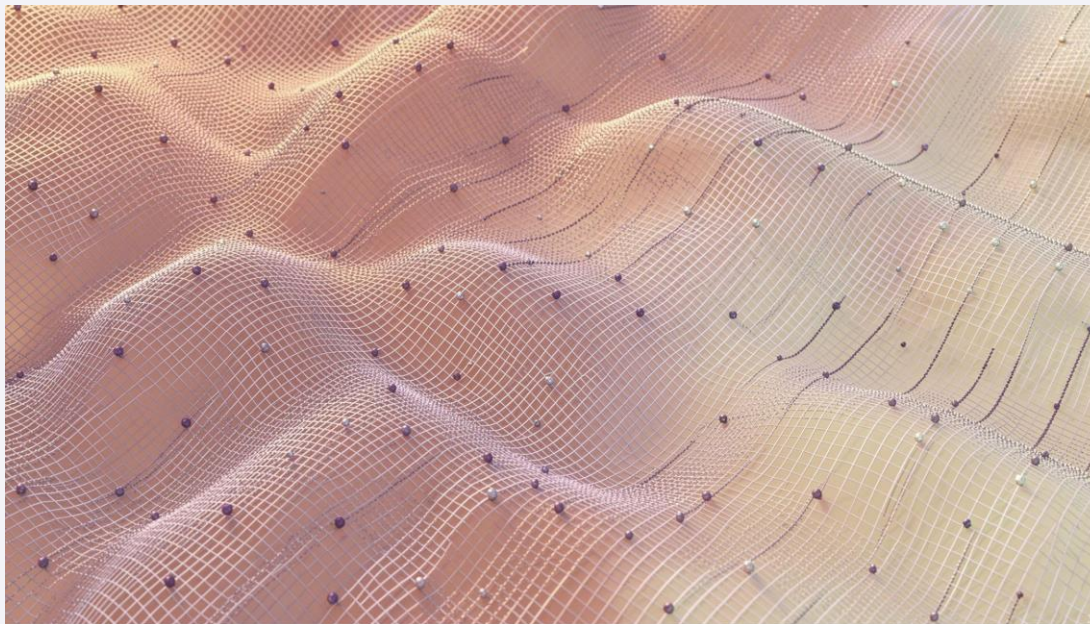
Deduplicate & Validate

Remove duplicate emails; filter invalid experience values

05

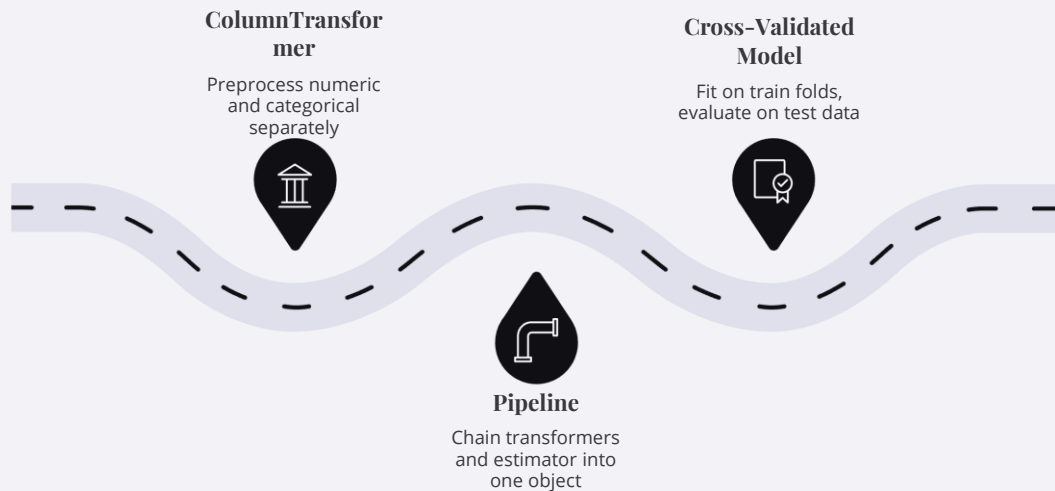
Engineer Features

Create `salary_ratio`, `tenure_days`, `is_senior` columns



Deliverable: A clean, feature-rich DataFrame — validated, de-duplicated, and ready to feed directly into a scikit-learn pipeline.

Scikit-learn Pipelines: End-to-End ML



Pipelines chain every step into a single object — **preventing data leakage** by ensuring transformers are fit only on training data, never on test data.

Key scikit-learn Concepts

Pipeline Object

Chain preprocessing + model into one callable — `pipe.fit(X_train, y_train)` handles everything.

ColumnTransformer

Apply `StandardScaler` to numerics and `OneHotEncoder` to categoricals in parallel.

No Data Leakage

Transformers fit only on `X_train`; test data is never seen during preprocessing.

Pandas: Data Manipulation & Cleaning

Pandas is the bridge between raw data and ML-ready features — your **first stop in every workflow**.

DataFrames

Tabular data structures — like Excel, but fully programmable in Python

Missing Values

Drop, fill, or interpolate nulls with `.dropna()` and `.fillna()`

GroupBy & Aggregation

Compute category-level stats: `df.groupby('region')['sales'].mean()`

5-Line Power

Load CSV → drop nulls → group → aggregate → export. Production-grade in minutes.

- ☐ The full ML stack in one sentence: **NumPy** computes fast, **Pandas** cleans and shapes, **scikit-learn** trains and deploys. Master these three and you own the workflow.

